MODULE 3

MQTT CONNECTIVITY WITH MICROPYTHON, RASPBERRY PC PICO W, AND ECLIPSE MOSQUITTO

OBJECTIVES

1) Understand the MQTT protocol.
2) Understand MQTT communication through a local broker.
3) Understand an IoT implementation using Python on a Raspberry Pi Pico W microcontroller.

REQUIREMENTS

1) Thonny as the MicroPython IDE,
2) Raspberry Pi Pico W,
3) Local server (PC Server).
4) Basic knowledge of Python programming.

3.1 MQTT

MQTT is a lightweight client-server message-transport protocol based on the publish-subscribe model and designed for environments with limited bandwidth, power, or device resources. Clients communicate through a broker using topics. MQTT supports several Quality of Service (QoS) levels, sessions, retained messages, and Last Will messages, depending on the protocol version and features used.

3.2 Advantages of MQTT

MQTT is widely used in IoT systems because of the following characteristics:

a. **Lightweight and Efficient**

MQTT can be implemented in IoT systems on small microcontrollers because its messages and protocol overhead can be compact. This helps optimize the use of network bandwidth.

b. **Scalable**

The broker architecture and separation between publishers and subscribers allow a system to be expanded to serve many clients. Actual scalability still depends on broker capacity, topic design, message size, QoS, the network, and connection patterns.

c. **Reliable**

MQTT provides QoS 0, QoS 1, and QoS 2 with different delivery guarantees. Reliability is not the same as latency: higher QoS levels may increase packet exchanges and overhead. The selected QoS should match the acceptable risk of message loss or duplication.

d. **Widely Supported**

MQTT is supported by many programming languages and platforms, including Python libraries specifically designed to implement the MQTT protocol on IoT devices. This broad support simplifies development across different devices and systems.

3.3 MQTT Communication Flow

As described previously, MQTT communication uses the **publish-subscribe** model to transfer data. Data exchanged between devices is carried in **messages** containing information such as sensor readings or commands.

3.3.1 Publish-Subscribe

The terms publish and subscribe describe how MQTT clients exchange messages through a broker. A publisher sends a message to a particular topic, while a subscriber uses a topic filter so that the broker forwards matching messages.

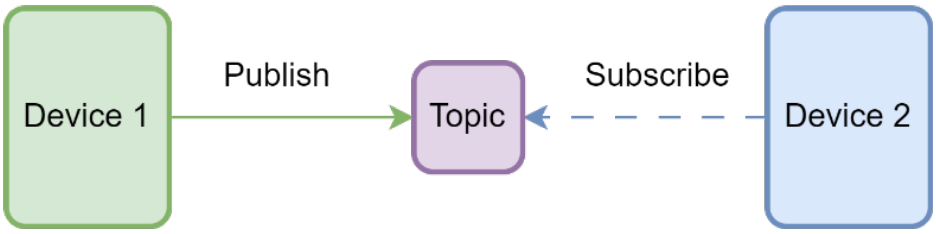


Figure 3.1: Publish-subscribe visualization

As shown in Figure 3.1, Device 1 publishes a message to a topic and Device 2 subscribes with a matching topic filter. The broker receives the message from Device 1 and forwards it to Device 2.

3.3.2 Topics

A topic is a hierarchical string used by the broker to route messages. A client publishes a message to a topic, while a subscriber uses a topic filter to receive matching messages. Topics are case-sensitive; the + and # wildcards may be used in subscriptions according to MQTT rules, but they are not used in topic names when publishing.

An MQTT topic is a **case-sensitive** string, so differences in capitalization produce different topic names.

3.3.3 Broker

The broker receives messages from publishers, matches them against subscriptions, and forwards them to clients whose topic filters match. Depending on its configuration, the broker may also manage sessions, QoS, retained messages, authentication, and authorization.

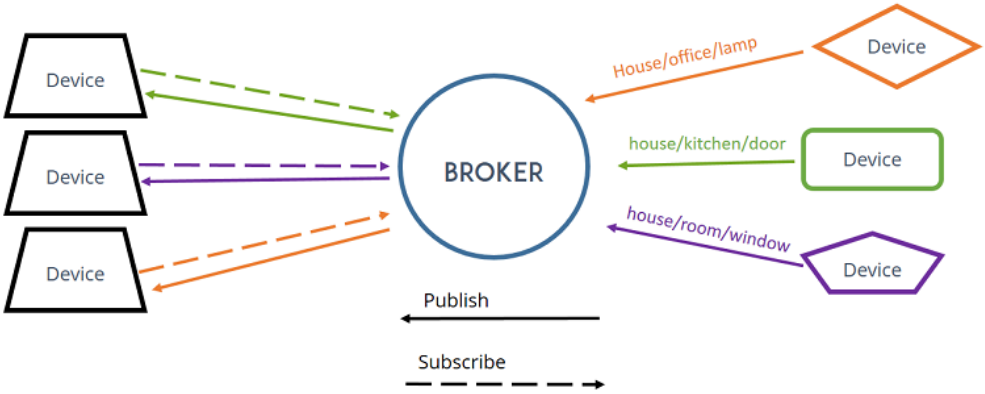


Figure 3.2: Broker visualization

Various MQTT broker services are available, including Adafruit IO and HiveMQ, with different service plans and usage limits. A broker can also be self-hosted on a single-board computer such as a PC using Mosquitto.

3.4 Eclipse Mosquitto

Eclipse Mosquitto is a lightweight, open-source MQTT message broker that can run on various systems, including the PC. Mosquitto makes it possible to set up a local broker without depending on an external broker service such as Adafruit IO and HiveMQ.



Figure 3.3: Mosquitto MQTT logo

3.5 MicroPython

MicroPython is an implementation of Python optimized to run on microcontrollers and low-power devices. Created by Damien George, MicroPython allows developers to use the Python programming language on hardware with limited resources, including microcontrollers and Internet of Things (IoT) devices.

The following are several key MicroPython features and characteristics:

a. **Lightweight and Optimized**

MicroPython is designed to run on hardware with limited resources. Its interpreter and Python language core are reduced in size to provide effective operation on microcontrollers with limited memory.

b. **Support for Popular Microcontrollers**

MicroPython supports a range of popular microcontrollers, including the ESP8266, ESP32, and STM32 families. This allows developers to use Python on hardware platforms commonly used in microcontroller projects.

c. **Interactive**

One of MicroPython's important features is its interactive mode. Users can interact directly with the hardware through the REPL (read-eval-print loop), which resembles Python on a desktop computer. This facilitates development and debugging.

d. **Support for Python-Compatible Modules**

Although it is designed for microcontrollers, MicroPython includes a number of modules that resemble modules available in other Python implementations. These modules may differ in size, features, or implementation, and not every CPython standard-library module is available without adaptation.

e. **I/O Capabilities**

MicroPython provides interfaces for accessing I/O pins and other microcontroller hardware, enabling developers to control sensors, motors, and other components.

f. **Wi-Fi and Networking Support**

Some MicroPython ports, particularly those for devices such as the ESP8266 and ESP32, support Wi-Fi and networking, allowing devices to connect to local networks or the internet.

g. **A Growing Community and Ecosystem**

MicroPython has an active developer community and many online resources and tutorials. These resources make it easier for developers to begin using MicroPython and troubleshoot problems.

MicroPython brings the accessibility and expressive power of Python to microcontrollers, making it an attractive option for IoT projects and low-power hardware development.

3.6 Raspberry Pi Pico W

The Raspberry Pi Pico W is an RP2040-based microcontroller board with a dual-core Arm Cortex-M0+ processor, 264 kB of SRAM, and 2 MB of external flash on the standard board. The Pico W adds 2.4 GHz IEEE 802.11 b/g/n Wi-Fi connectivity and Bluetooth 5.2 support through a wireless chip that communicates with the RP2040. The board can be programmed using MicroPython, C, or C++; MicroPython is not the only supported programming language. The pinout diagram is shown in Appendix 3.

3.7 Implementing the MQTT Protocol

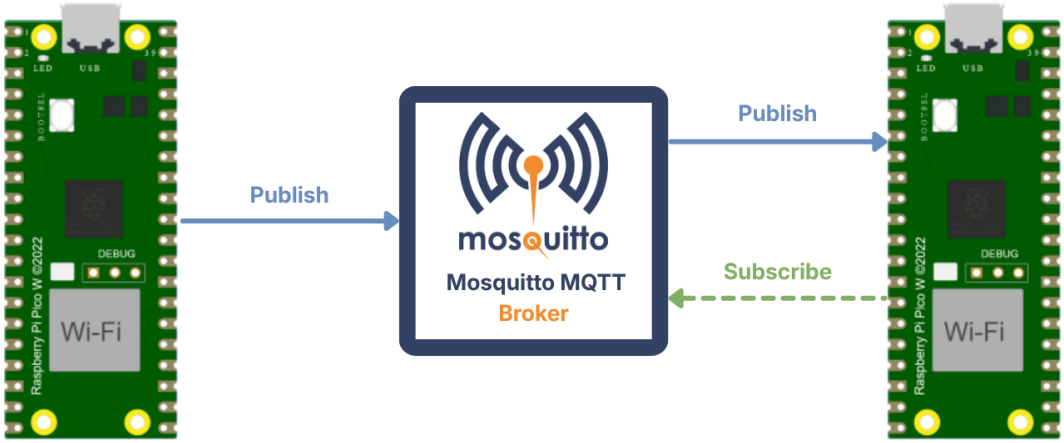


Figure 3.4: MQTT implementation with the Raspberry Pi Pico W

The message flow between the Raspberry Pi Pico W and Mosquitto MQTT is as follows:

a. Device 1 publishes a message to topic 'A' on the MQTT broker.
b. Device 2 subscribes to topic 'A'.
c. The MQTT broker forwards the message from Device 1 to Device 2 according to the subscription.

3.7.1 Setting Up a Local Broker on the PC

MQTT requires a broker to act as an intermediary between clients that publish messages and clients that subscribe. In this practicum, a PC single-board computer runs the local MQTT broker. This section is optional when an external broker such as HiveMQ or Adafruit IO is used.

Install the `mosquitto` packages on the PC by running the following commands in the terminal:

Installing the Mosquitto MQTT Broker

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

When a command is run with `sudo`, the system may request the password for the PC user account. The `mosquitto` and `mosquitto-clients` packages are used to configure and test the MQTT broker.

Next, run the following command in the terminal to enable Mosquitto automatically when the PC starts:

Enabling Mosquitto MQTT at Startup

```
sudo systemctl enable mosquitto.service
```

Run the following command in the terminal to view the PC IP address:

Viewing the PC IP Address

```
hostname -I
# Alternatively, use the following command
ip addr
```

The commands above display the PC's IPv4 and IPv6 addresses. Only the IPv4 address is required for the broker in this example, such as `192.168.1.100`.

3.7.1.1 Without Authentication

To allow other devices to access the broker without authentication, edit the Mosquitto configuration. Open `/etc/mosquitto/mosquitto.conf` with a text editor, for example `nano` in the terminal:

Opening a Text Editor in the Terminal

```
sudo nano /etc/mosquitto/mosquitto.conf
```

In editing mode, add the following options at the end of the `mosquitto.conf` configuration:

Contents of the mosquitto.conf Configuration File

```
# Default configuration
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

# New file content
listener 1883
allow_anonymous true
```

In the configuration above, `listener 1883` sets the MQTT listener port, while `allow_anonymous true` allows any client to use the broker without authentication. This configuration must only be used on an isolated practicum network and must not be exposed to the internet. After saving the configuration, restart the Mosquitto service.

Restarting the Mosquitto MQTT Service

```
sudo systemctl restart mosquitto.service
```

3.7.2 With Authentication

To add authentication to the broker, several additional configuration steps are required. Create the username and password that will be used by MQTT clients.

Adding a Username and Password to Mosquitto MQTT

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd YOUR_USERNAME
# After running the command above, enter the password when prompted.

# Change the file permissions so that Mosquitto can read the file
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

When entering a password, the terminal normally does not display the typed characters. Record and store the password securely. Then change the default `mosquitto.conf` configuration as follows:

Contents of the mosquitto.conf Configuration File

```
# New file content
# Authentication configuration applies to this listener

# Default configuration
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

```

```
# New file content
allow_anonymous false
listener 1883
password_file /etc/mosquitto/passwd
```

Unlike the previous configuration, `allow_anonymous` is set to `false` so that clients without valid credentials cannot use the broker. Username-and-password authentication does not encrypt traffic; use TLS when the broker is accessed over an untrusted network. Next, ensure that Mosquitto can read the `passwd` file by running the following commands:

Changing the passwd File Permissions

```
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

Finally, restart the Mosquitto service.

Restarting the Mosquitto MQTT Service

```
sudo systemctl restart mosquitto.service
```

3.7.3 Setting Up the MQTT Library on the Raspberry Pi Pico W

The library used to run MQTT on the Pico W is `micropython-umqtt.simple`. The following steps show how to add the library through Thonny for MicroPython:

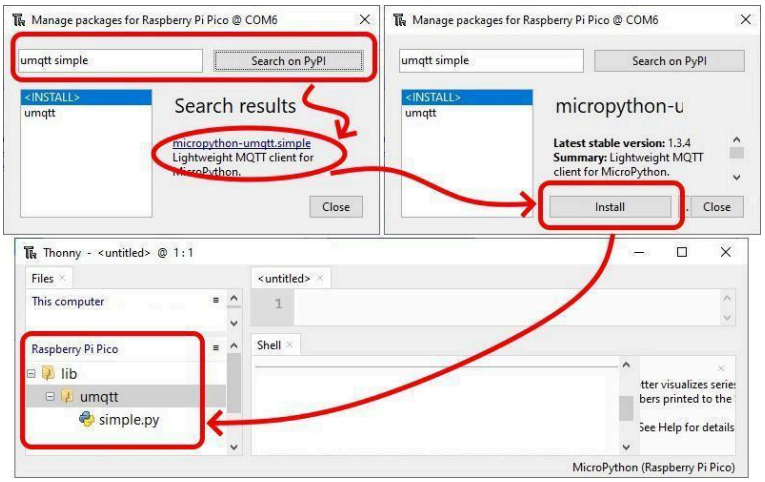


Figure 3.5: Tools > Manage Packages

3.7.4 Raspberry Pi Pico W Source Code

3.7.4.1 Publish

a. **Required Libraries and Variables**

The library required to use the MQTT protocol in MicroPython is `umqtt.simple`.

MicroPython Library Import

```
import network                      # Network connectivity
import time                         # Provides sleep()
from umqtt.simple import MQTTClient # MQTT library
import utime                        # Time formatting
```

After importing the libraries, declare the required variables. These include:

- Variables for the Wi-Fi network
- Variables for the MQTT client

- A variable containing the data to be published to the broker

Source Code for Variable Initialization

```
# Enter the Wi-Fi name and password for the network shared with the local broker
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Enter the address of the broker being used (Mosquitto, HiveMQ, Adafruit IO)
# or another broker
mqtt_host     = "<MQTT HOST ADDRESS>"

# Optional: credentials when authentication is enabled
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Enter the topic to which messages will be published
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Enter a client ID that is unique for each client
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Number to publish
counter = 0
```

b. Connecting the Pico W to Wi-Fi and the MQTT Server

After declaring the variables, connect the Pico W to Wi-Fi and the MQTT server. The function used to connect to Wi-Fi is:

```
wlan.connect("<YOUR WIFI SSID>", "<YOUR WIFI PASSWORD>")
```

and the function used to connect the MQTT client is:

```
mqtt_client.connect()
```

Source Code for Connecting to Wi-Fi and the MQTT Server

```
# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Optional: use when authentication is enabled
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()
```

c. The `try` Block

In the code below, the Pico W publishes the value stored in the `counter` variable using:

```
mqtt_client.publish("<Target topic>", str(<Value to publish>))
```

Source Code for the try Block (Comparable to the Arduino loop Function)

```
try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Handle transmission errors
except Exception as e:
  print(f'MQTT error: {e}')

# Disconnect when the Python program stops
finally:
  mqtt_client.disconnect()
```

The information is repeatedly published to the MQTT server every three seconds through `time.sleep(3)`.

3.7.4.2 Subscribe

As in the publisher example, the library required to subscribe to MQTT topics in MicroPython is `umqtt.simple`.

MicroPython Library Import

```
import network
import time
from umqtt.simple import MQTTClient
import utime
```

Source Code for Variable Initialization

```
# Enter the Wi-Fi name and password for the network shared with the local broker
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Enter the address of the broker being used (Mosquitto, HiveMQ, Adafruit IO)
# or another broker
mqtt_host     = "<MQTT HOST ADDRESS>"

# Optional: credentials when authentication is enabled
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Enter the topic filter to subscribe to
mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"

# Enter a client ID that is unique for each client
mqtt_client_id = "iij0dfq8f012nif2jnfijn"
```

Source Code for Connecting to Wi-Fi and the MQTT Server

```
# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Optional: use when authentication is enabled
  user=mqtt_username,
  password=mqtt_password
)
```

The main difference between the subscriber and publisher source code is the callback function, which is invoked whenever a new message arrives on a subscribed topic.

MQTT Callback Function for a New Message

```
def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
    current_time[3], current_time[4], current_time[5]
  )
  # Debug print out of what was received over MQTT
  print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()
```

After defining the callback function, subscribe to the required topic filter using:

```
mqtt_client.subscribe("<Topic_yang_dilanggani>")
```

Subscribing to the Selected Topic

```
# Once connected, subscribe to the MQTT topic
mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")
```

Source Code for the try Block (Comparable to the Arduino loop Function)

```
try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Handle transmission errors
except Exception as e:
  print(f'MQTT error: {e}')
```

```
12
13  # Disconnect when the Python program stops
14  finally:
15    mqtt_client.disconnect()
```

In the code above, `mqtt_client.wait_msg()` waits for a message from an MQTT client publishing to the subscribed topic. When data is received, the program produces output according to the behavior defined in `mqtt_subscription_callback()`.

APPENDIX: MODULE 3

Source Code for Publishing MQTT Messages to Mosquitto

Publish MQTT Messages to Mosquitto

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Enter the Wi-Fi name and password for the network shared with the local broker
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Enter the address of the broker being used (Mosquitto, HiveMQ, Adafruit IO)
# or another broker
mqtt_host     = "<MQTT HOST ADDRESS>"

# Optional: credentials when authentication is enabled
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Enter the topic to which messages will be published
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Enter a client ID that is unique for each client
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Number to publish
counter = 0

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Optional: use when authentication is enabled
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()

try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
```

```
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Handle transmission errors
except Exception as e:
  print(f'MQTT error: {e}')

# Disconnect when the Python program stops
finally:
  mqtt_client.disconnect()
```

Source Code for Subscribing to MQTT Messages from Mosquitto

Subscribe to MQTT Messages from Mosquitto

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Enter the Wi-Fi name and password for the network shared with the local broker
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Enter the address of the broker being used (Mosquitto, HiveMQ, Adafruit IO)
# or another broker
mqtt_host     = "<MQTT HOST ADDRESS>"

# Optional: credentials when authentication is enabled
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Enter the topic filter to subscribe to
mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"

# Enter a client ID that is unique for each client
mqtt_client_id = "iij0dfq8f012nif2jnfijn"

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Optional: use when authentication is enabled
  user=mqtt_username,
  password=mqtt_password
)

def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
```

```
      current_time[3], current_time[4], current_time[5]
    )
    # Debug print out of what was received over MQTT
    print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()

mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")

try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Handle transmission errors
except Exception as e:
  print(f'MQTT error: {e}')

# Disconnect when the Python program stops
finally:
  mqtt_client.disconnect()
```